



GLORYBELLE

GIOIELLERIA ITALIANA

Backend Technical Design Document

Database Schema · API Design

1. Overview

This document specifies the backend data model and REST API for the GLORYBELLE e-commerce platform. It covers the full customer journey implemented in the frontend prototype — category browsing, product filtering, the two-image hover card, wishlist, cart with metal/size variant selection, guest and authenticated checkout, Stripe payment, and Italian electronic invoicing (Fatturazione Elettronica).

Stack: Django 5 + Django REST Framework · PostgreSQL 16 (AWS RDS, eu-south-1 Milan) · Redis (sessions/cache) · Celery (async jobs) · Stripe (payments) · Fattura24 API (e-invoicing) · SendGrid (email).

Scope of this document: data model design and API contract only. Deployment topology, CI/CD, and frontend implementation are covered separately.

2. Database Schema

Seventeen models across six functional areas: identity, catalog, cart, commerce, fulfillment, and content. Each table below lists fields in the order they would appear in the Django model.

User

Django's built-in auth model. Email is used as the effective login identifier.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
email	EmailField	unique, used for login
username	CharField(150)	unique
password	CharField	hashed by Django
first_name / last_name	CharField(150)	
is_active	Boolean	default True
date_joined	DateTimeField	auto_now_add

CustomerProfile

One-to-one extension of User with e-commerce-specific fields.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
user	OneToOneField(User)	on_delete=CASCADE
phone	CharField(20)	nullable
newsletter_opt_in	Boolean	default False
preferred_language	CharField(5)	default 'it'
created_at	DateTimeField	auto_now_add

Address

Shipping/billing addresses. A customer may store several.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
user	ForeignKey(User)	CASCADE, related_name='addresses'
full_name	CharField(150)	
address_line1 / 2	CharField(255)	line2 nullable
city	CharField(100)	
postal_code	CharField(10)	
province	CharField(2)	Italian province code
country	CharField(2)	default 'IT'
phone	CharField(20)	
is_default_shipping	Boolean	default False
is_default_billing	Boolean	default False

Category

Product taxonomy that powers the mega-menu and shop filters.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
name	CharField(100)	e.g. 'Anelli'
slug	SlugField	unique, e.g. 'rings'
parent	ForeignKey('self')	nullable, for subcategories
display_order	IntegerField	default 0
icon_svg	TextField	nullable, mega-menu icon markup

Product

The parent product record — information shared across all metal/size variants.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
name	CharField(200)	
slug	SlugField	unique
description	TextField	
category	ForeignKey(Category)	PROTECT, related_name='products'
gem	CharField(20)	choices: diamond / emerald / ruby / sapphire / amber
purity	CharField(10)	e.g. '750', '925'
base_price	DecimalField(10,2)	EUR
old_price	DecimalField(10,2)	nullable — strike-through comparison price
sku_prefix	CharField(20)	unique
is_active	Boolean	default True
is_featured	Boolean	default False — trending carousel
created_at / updated_at	DateTimeField	auto_now_add / auto_now

ProductVariant

The actual sellable unit: one specific metal + size combination, with its own stock and SKU.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
product	ForeignKey(Product)	CASCADE, related_name='variants'
metal	CharField(10)	choices: gold / rose / silver
size	CharField(10)	e.g. '52', 'M', '45 cm'
sku	CharField(40)	unique
price_override	DecimalField(10,2)	nullable — overrides product.base_price
stock_quantity	IntegerField	default 0 — physical stock on hand
reserved_quantity	IntegerField	default 0 — held by open carts, see §6.11
is_active	Boolean	default True

StockReservation

One active cart-hold on a variant. Expires automatically if checkout isn't completed in time.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
variant	ForeignKey(ProductVariant)	CASCADE, related_name='reservations'
cart_item	OneToOneField(CartItem)	CASCADE
quantity	PositiveIntegerField	
expires_at	DateTimeField	indexed — now() + 15 minutes at creation
created_at	DateTimeField	auto_now_add

ProductImage

Ordered images per product. display_order 0 is primary, 1 is the hover-swap image.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
product	ForeignKey(Product)	CASCADE, related_name='images'
image	URLField	S3 / Cloudinary asset URL
alt_text	CharField(200)	
display_order	IntegerField	default 0

Cart

One active cart per logged-in user, or per guest session.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
user	ForeignKey(User)	nullable, CASCADE, related_name='carts'
session_key	CharField(40)	nullable — identifies a guest cart
created_at / updated_at	DateTimeField	auto_now_add / auto_now

CartItem

A single line item inside a cart.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
cart	ForeignKey(Cart)	CASCADE, related_name='items'
variant	ForeignKey(ProductVariant)	PROTECT
quantity	PositiveIntegerField	default 1
added_at	DateTimeField	auto_now_add

Wishlist / WishlistItem

One wishlist per user; *WishlistItem* links a saved *Product* to it.

Field	Type	Constraints / Notes
Wishlist.id	BigAutoField	Primary key
Wishlist.user	OneToOneField(User)	CASCADE
WishlistItem.wishlist	ForeignKey(Wishlist)	CASCADE, related_name='items'
WishlistItem.product	ForeignKey(Product)	CASCADE
WishlistItem.added_at	DateTimeField	auto_now_add

Order

A placed order. Guest checkout is supported via a nullable user field.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
order_number	CharField(20)	unique, e.g. 'AUR-2026-00042'
idempotency_key	CharField(64)	unique, indexed — see §6.3
user	ForeignKey(User)	nullable, SET_NULL, related_name='orders'
email	EmailField	for guest orders / confirmation email
status	CharField(20)	indexed — pending / paid / processing / shipped / delivered / cancelled / refunded
shipping_address	ForeignKey(Address)	PROTECT
billing_address	ForeignKey(Address)	PROTECT
subtotal / shipping_cost	DecimalField(10,2)	shipping_cost default 0
tax_amount	DecimalField(10,2)	IVA — 22% standard rate
total	DecimalField(10,2)	
currency	CharField(3)	default 'EUR'
stripe_payment_intent_id	CharField(100)	nullable, indexed
placed_at / updated_at	DateTimeField	auto_now_add / auto_now, indexed

ProcessedWebhookEvent

Deduplication ledger for Stripe webhooks — Stripe guarantees at-least-once delivery, so the same event can arrive more than once.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
stripe_event_id	CharField(80)	unique, indexed
event_type	CharField(60)	e.g. 'payment_intent.succeeded'
processed_at	DateTimeField	auto_now_add

OrderItem

A line item inside an order. Key values are snapshotted at purchase time.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
order	ForeignKey(Order)	CASCADE, related_name='items'
variant	ForeignKey(ProductVariant)	PROTECT
product_name_snapshot	CharField(200)	
metal_snapshot / size_snapshot	CharField(10)	
price_snapshot	DecimalField(10,2)	price at time of purchase
quantity	PositiveIntegerField	
line_subtotal	DecimalField(10,2)	

Payment

Stripe payment record, one-to-one with an Order.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
order	OneToOneField(Order)	CASCADE, related_name='payment'
stripe_payment_intent_id	CharField(100)	unique
stripe_charge_id	CharField(100)	nullable
amount	DecimalField(10,2)	
currency	CharField(3)	default 'EUR'
status	CharField(20)	requires_payment / succeeded / failed / refunded
paid_at	DateTimeField	nullable

Invoice

Fatturazione Elettronica (SDI) record via the Fattura24 API, one-to-one with an Order.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
order	OneToOneField(Order)	CASCADE, related_name='invoice'
invoice_number	CharField(30)	unique
fattura24_id	CharField(50)	external reference ID
pdf_url	URLField	nullable
status	CharField(20)	pending / generated / sent / error
issued_at	DateTimeField	nullable

Review

Product review / testimonial, shown after moderation.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
product	ForeignKey(Product)	nullable, CASCADE, related_name='reviews'
user	ForeignKey(User)	nullable, SET_NULL
customer_name / customer_location	CharField(100)	e.g. 'Milano'
rating	PositiveSmallIntegerField	1–5
comment	TextField	
is_approved	Boolean	default False — moderation gate
created_at	DateTimeField	auto_now_add

NewsletterSubscriber

Footer newsletter signup.

Field	Type	Constraints / Notes
id	BigAutoField	Primary key
email	EmailField	unique
subscribed_at	DateTimeField	auto_now_add
is_active	Boolean	default True

2.1 Key Relationships

User 1—1 CustomerProfile · User 1—N Address · User 1—N Order (nullable, SET_NULL)
Category 1—N Product · Product 1—N ProductVariant · Product 1—N ProductImage
Cart 1—N CartItem —N:1— ProductVariant · User/session 1—1 active Cart
User 1—1 Wishlist 1—N WishlistItem —N:1— Product
Order 1—N OrderItem —N:1— ProductVariant · Order 1—1 Payment · Order 1—1 Invoice
Order N:1 Address (shipping) · Order N:1 Address (billing) — both PROTECT
Product 1—N Review (moderated via is_approved)
CartItem 1—1 StockReservation —N:1— ProductVariant (\$6.11)

3. API Design

RESTful JSON API under `/api/`. Authentication uses JWT bearer tokens (access + refresh) via `djangorestframework-simplejwt`. "Optional" auth means the endpoint works for both guest sessions and logged-in users.

Authentication

Method	Endpoint	Auth	Description
POST	<code>/api/auth/register/</code>	No	Create a new customer account.
POST	<code>/api/auth/login/</code>	No	Obtain a JWT token pair.
POST	<code>/api/auth/token/refresh/</code>	No	Exchange a refresh token for a new access token.
POST	<code>/api/auth/logout/</code>	Yes	Blacklist the current refresh token.
POST	<code>/api/auth/password-reset/</code>	No	Send a password-reset email.
GET	<code>/api/auth/me/</code>	Yes	Get the current user's profile.
PATCH	<code>/api/auth/me/</code>	Yes	Update profile fields.

Endpoint	Request Body	Response
POST <code>/api/auth/register/</code>	{email, password, first_name, last_name}	{user, access, refresh}
POST <code>/api/auth/login/</code>	{email, password}	{access, refresh}
POST <code>/api/auth/token/refresh/</code>	{refresh}	{access}
POST <code>/api/auth/logout/</code>	{refresh}	204 No Content
POST <code>/api/auth/password-reset/</code>	{email}	202 Accepted
GET <code>/api/auth/me/</code>	—	{user, profile}
PATCH <code>/api/auth/me/</code>	{phone, newsletter_opt_in, ...}	{profile}

Categories

Method	Endpoint	Auth	Description
GET	<code>/api/categories/</code>	No	List categories for the mega-menu / shop filters.

Endpoint	Request Body	Response
GET <code>/api/categories/</code>	—	[[{id, name, slug, icon_svg}]]

Products

Method	Endpoint	Auth	Description
GET	<code>/api/products/</code>	No	List products — filterable & paginated.
GET	<code>/api/products/{slug}/</code>	No	Product detail with variants and images.
GET	<code>/api/products/featured/</code>	No	Featured products for the trending carousel.
GET	<code>/api/products/{slug}/reviews/</code>	No	List approved reviews for a product.

Method	Endpoint	Auth	Description
POST	/api/products/{slug}/reviews/	Yes	Submit a review (queued for moderation).

Endpoint	Request Body	Response
GET /api/products/	Query: category, metal, gem, price_min, price_max, search, ordering, page	{count, next, previous, results}
GET /api/products/{slug}/	—	{product, variants, images}
GET /api/products/featured/	—	[[product]]
GET /api/products/{slug}/reviews/	—	[[review]]
POST /api/products/{slug}/reviews/	{rating, comment}	{review}

Cart

Method	Endpoint	Auth	Description
GET	/api/cart/	Optional	Get the current cart (by user or session).
POST	/api/cart/items/	Optional	Add an item to the cart.
PATCH	/api/cart/items/{id}/	Optional	Update an item's quantity.
DELETE	/api/cart/items/{id}/	Optional	Remove one item.
DELETE	/api/cart/	Optional	Clear the whole cart.

Endpoint	Request Body	Response
GET /api/cart/	—	{items, subtotal}
POST /api/cart/items/	{variant_id, quantity}	{cart_item}
PATCH /api/cart/items/{id}/	{quantity}	{cart_item}
DELETE /api/cart/items/{id}/	—	204 No Content
DELETE /api/cart/	—	204 No Content

Wishlist

Method	Endpoint	Auth	Description
GET	/api/wishlist/	Yes	Get wishlist items.
POST	/api/wishlist/items/	Yes	Add a product to the wishlist.
DELETE	/api/wishlist/items/{product_id}/	Yes	Remove a product from the wishlist.

Endpoint	Request Body	Response
GET /api/wishlist/	—	[[product]]
POST /api/wishlist/items/	{product_id}	{wishlist_item}
DELETE /api/wishlist/items/{product_id}/	—	204 No Content

Addresses

Method	Endpoint	Auth	Description
GET	/api/addresses/	Yes	List the customer's saved addresses.
POST	/api/addresses/	Yes	Add a new address.
PATCH	/api/addresses/{id}/	Yes	Update an address.
DELETE	/api/addresses/{id}/	Yes	Delete an address.

Endpoint	Request Body	Response
GET /api/addresses/	—	[[address]]
POST /api/addresses/	{full_name, address_line1, ...}	{address}
PATCH /api/addresses/{id}/	{...}	{address}
DELETE /api/addresses/{id}/	—	204 No Content

Checkout & Orders

Method	Endpoint	Auth	Description
POST	/api/checkout/	Optional	Create an Order + Stripe PaymentIntent.
POST	/api/webhooks/stripe/	Stripe signature	Handle payment success/failure events.
GET	/api/orders/	Yes	List the customer's order history.
GET	/api/orders/{order_number}/	Yes	Order detail, items, and invoice.

Endpoint	Request Body	Response
POST /api/checkout/	{shipping_address, billing_address, email}	{order, client_secret}
POST /api/webhooks/stripe/	Stripe event payload	200 OK
GET /api/orders/	—	[[order_number, status, total, placed_at]]
GET /api/orders/{order_number}/	—	{order, items, invoice}

Newsletter & Search

Method	Endpoint	Auth	Description
POST	/api/newsletter/subscribe/	No	Subscribe an email to the newsletter.
GET	/api/search/	No	Quick product search (mega-menu / header).

Endpoint	Request Body	Response
POST /api/newsletter/subscribe/	{email}	201 Created
GET /api/search/	Query: q	[[product]]

4. Senior-Level Design Decisions

Reasoning behind the non-obvious choices in this schema, for future maintainers.

Product vs. ProductVariant split

Metal and size create real SKU-level differences — different stock, occasionally different price. Splitting them lets inventory be tracked accurately per combination instead of guessing stock at the product level.

Guest cart via session_key

The cart works before login using Django's session key. On login, the session cart's items are merged into the user's persistent cart so nothing gets lost.

Snapshot fields on OrderItem

product_name_snapshot, metal_snapshot, price_snapshot, etc. exist because a product's name or price can change after an order is placed. Order history must always reflect exactly what was purchased — never the live product record.

PROTECT vs. CASCADE vs. SET_NULL

PROTECT is used on the ProductVariant and Address references from CartItem / Order so a variant or address can never be deleted while an order still points to it. SET_NULL is used on Order.user so a customer account can be deleted (GDPR 'right to be forgotten') without deleting their order records.

Order status as a single field

A CharField with choices is sufficient at this scale. A dedicated state-machine library (e.g. django-fsm) is worth introducing later once fulfillment logic grows more complex — partial shipments, returns, exchanges.

Stripe PaymentIntent flow

An Order is created in 'pending' status together with a Stripe PaymentIntent. The Stripe webhook — not the browser redirect — is the source of truth for marking an order 'paid'. This avoids trusting the client and correctly handles a user closing the tab before the confirmation page loads.

Invoice as its own model

Decoupling e-invoicing from Order lets the async Celery task retry independently if the Fattura24 API is briefly unavailable, without blocking order confirmation to the customer.

5. Product Filter Query Parameters

Reference for GET /api/products/, matching the shop sidebar filters (category, metal, gem, price) and sort dropdown in the frontend.

Parameter	Values	Notes
category	slug	e.g. rings, necklaces, earrings, bracelets
metal	gold rose silver	
gem	diamond emerald ruby sapphire amber	
price_min / price_max	decimal	EUR bounds
search	string	matches name and description
ordering	-created_at price -price name	default: -created_at
page / page_size	integer	pagination, default page_size=8

6. Scalability, Concurrency & Performance

The schema in §2 is a correct *logical* design. On its own it does not guarantee safe or fast behavior under concurrent load — that requires the specific measures in this section, targeting roughly 5,000 concurrent users including simultaneous checkout traffic.

6.1 Realistic Performance Targets

No system — including Stripe or Amazon — returns literally identical response times at 1 user and at 5,000 concurrent users; resource contention is real. The correct goal is **low, predictable latency under load**, measured with percentiles rather than a single average:

Metric	Product browsing / search	Checkout & payment
p50 (median)	< 80 ms	< 120 ms
p95	< 200 ms	< 350 ms
p99	< 400 ms	< 800 ms
Error rate	0%	< 0.1% (retried safely)

These targets are only meaningful once verified with actual load testing (Locust or k6 simulating 5,000 concurrent virtual users) against a staging environment sized like production — they are a design target here, not yet a measured result.

6.2 Preventing Stock Overselling

The highest-risk gap in a naive implementation: if 5,000 users hit “buy” on the last unit of a variant at the same moment, a plain read `stock` → `check` → `subtract` sequence lets many requests read the same stock value before any of them writes back — all can succeed, leaving stock negative. The fix is a row lock inside a database transaction so concurrent requests for the same variant are serialized:

```
with transaction.atomic():
    variant = (ProductVariant.objects
               .select_for_update()
               .get(pk=variant_id))
    if variant.stock_quantity < quantity:
        raise OutOfStock(variant)
    variant.stock_quantity -= quantity
    variant.save(update_fields=["stock_quantity"])
    # ... create OrderItem within the same transaction
```

`select_for_update()` locks only the one variant row being purchased, so shoppers buying *different* products are never blocked by each other — contention only happens on the same popular item, which is the correct and minimal amount of locking.

6.3 Payment Idempotency

A network retry or an impatient double-click on “Pay” must never create two charges for one order. The frontend generates one UUID per checkout attempt and sends it as `Order.idempotency_key`; the same value is passed to Stripe as the `PaymentIntent`’s idempotency key. Stripe then guarantees that retried requests with the same key return the original `PaymentIntent` instead of creating a new charge, and a unique constraint on `idempotency_key` gives the same guarantee at the database level.

```
stripe.PaymentIntent.create(
    amount=order.total_in_cents,
    currency="eur",
    idempotency_key=order.idempotency_key,
)
```

6.4 Idempotent Webhook Handling

Stripe delivers webhooks at-least-once — the same `payment_intent.succeeded` event can arrive twice. The `ProcessedWebhookEvent` table (§2) records each Stripe event ID the first time it is handled; the handler checks that table before doing any work.

```
event_id = stripe_event["id"]
if ProcessedWebhookEvent.objects.filter(stripe_event_id=event_id).exists():
    return HttpResponse(status=200) # already handled, no-op

with transaction.atomic():
    ProcessedWebhookEvent.objects.create(stripe_event_id=event_id, ...)
    mark_order_paid(order)
```

6.5 Database Indexing Strategy

Every column used in a WHERE, JOIN, or ORDER BY on a hot path needs an index, or PostgreSQL falls back to a full table scan once the tables have real volume.

Column	Index type	Why
Product.slug	unique index	product detail page lookup
Product.category_id	index	shop filter by category
ProductVariant.sku	unique index	cart / order lookups
Order.status	index	admin dashboard, "my orders"
Order.idempotency_key	unique index	§6.3
Order.stripe_payment_intent_id	index	webhook lookup by intent ID
CartItem.cart_id	index (FK default)	render cart
ProcessedWebhookEvent.stripe_event_id	unique index	§6.4

6.6 Connection Pooling

Django opens one DB connection per request by default. PostgreSQL's default connection ceiling (~100) is exhausted quickly once multiple app server instances each hold their own pool at 5,000 concurrent users — the database starts rejecting connections rather than slowing down gracefully. **PgBouncer** in transaction-pooling mode sits between Django and PostgreSQL, multiplexing thousands of client connections onto a small, fixed number of real database connections.

6.7 Caching Strategy

Category lists and product listings are read far more often than they change. They are cached in **Redis** (list: 5 min TTL, product detail: 10 min TTL) and explicitly invalidated by a Django signal on Product/Category save — this removes the majority of read traffic from PostgreSQL entirely, leaving the database's capacity for cart and checkout writes.

6.8 Async Task Offloading

Nothing the customer is not directly waiting for should run inside the request/response cycle. Order confirmation email, the Fattura24 invoice call, and newsletter sends all run as **Celery** tasks queued from the webhook handler — so checkout latency (§6.1) is not affected by a slow third-party API, and a temporary SendGrid or Fattura24 outage is retried automatically instead of failing the order.

6.9 Horizontal Scaling & Read Replicas

Django app servers are stateless (session state lives in Redis, not in-process) so they scale horizontally behind a load balancer with no code change. At sustained high read volume, a PostgreSQL read replica can serve product browsing/search while the primary handles cart and order writes exclusively — worth adding once monitoring shows the primary's read load, not on day one.

6.10 Rate Limiting

DRF throttle classes cap request rate per user/IP on sensitive endpoints (login, checkout, review submission) so a single bad actor or buggy client script cannot degrade service for everyone else — e.g. 20 requests/minute on `/api/checkout/`, 5/minute on `/api/auth/login/`.

6.11 Stock Reservation (Cart-Hold) Pattern

§6.2's row lock guarantees only one buyer can ever complete the purchase of the last unit — but it does not, by itself, give the other shoppers an honest “in stock” signal *before* they reach checkout. On a single-piece luxury item, three customers can otherwise all see “available”, add it to their cart, and only the one who happens to click Pay first finds out the other two were shown false hope. High-demand retail (ticketing, limited drops) solves this with a short-lived **reservation**, not just a lock at the final instant.

Available stock shown to shoppers is `stock_quantity - reserved_quantity`, never the raw stock figure:

```
# Adding to cart – atomically reserve for RESERVATION_MINUTES
with transaction.atomic():
    variant = ProductVariant.objects.select_for_update().get(pk=variant_id)
    available = variant.stock_quantity - variant.reserved_quantity
    if available < quantity:
        raise OutOfStock(variant)          # honest, immediate rejection

    variant.reserved_quantity += quantity
    variant.save(update_fields=["reserved_quantity"])

    StockReservation.objects.create(
        variant=variant, cart_item=cart_item, quantity=quantity,
        expires_at=timezone.now() + timedelta(minutes=15),
    )
```

A Celery periodic task (every minute) releases expired holds automatically, so an abandoned cart doesn't lock stock away forever:

```
@shared_task
def release_expired_reservations():
    expired = StockReservation.objects.filter(expires_at__lt=timezone.now())
    for res in expired.select_related("variant"):
        with transaction.atomic():
            v = ProductVariant.objects.select_for_update().get(pk=res.variant_id)
            v.reserved_quantity = F("reserved_quantity") - res.quantity
            v.save(update_fields=["reserved_quantity"])
            res.delete()
```

Worked example — stock = 1, three shoppers A, B, C: A adds the item to their cart and reserved_quantity becomes 1, so B and C immediately see “sold out” on page refresh, not just at checkout. If A completes payment, the hold converts to a permanent -1 on stock_quantity. If A does nothing for 15 minutes, the Celery task releases the hold and the item becomes available to B and C again — whichever of them acts first now gets the same honest, immediate feedback.